DELTA – Střední škola informatiky a ekonomie, s.r.o.
Ke Kamenci 151, Pardubice

LifeMeter

Mobilní aplikace pro sledování zdravotních metrik

Autor: Zadrobílek Jan
Třída: 4.B
Studijní obor: Informační technologie 18-20-M/01
Školní rok: 2025/2026

Zadání maturitního projektu z informatických předmětů

Jméno a příjmení:	*Jan Zadrobílek*
Školní rok:	*2025/2026*
Třída:	*4.B*
Obor:	*Informační technologie 18-20-M/01*
Téma práce:	*Mobilní aplikace „LifeMeter“*
Vedoucí práce:	*Ing. Monika Borkovcová, Ph.D.*

Způsob zpracování, cíle práce, pokyny k obsahu a rozsahu práce:

Cílem projektu je vytvoření mobilní aplikace pro OS Android, která bude sloužit jako nástroj pro sledování fyzické aktivity, nutričních hodnot a spánku. Funkce budou zahrnovat možnost přihlášení, notifikace a lokální manipulaci s daty. Aplikace bude navržena tak, aby poskytovala uživatelům intuitivní uživatelské rozhraní a efektivní způsob interakce. Aplikace je funkční pro OS Android 10 a vyšší.

Popis funkčnosti mobilní aplikace, která umožňuje:

Záznam fyzických aktivit

- Možnost přidávat aktivity s informacemi jako poloha nebo poznámka.

Sledování výživy

- Přidávání zkonzumovaného jídla, doplňků stravy, sledování kalorií, makroživin a vitamínů.

Sledování spánku

- Záznam délky spánku s možností přidání poznámek, případně možnost pracovat s kvalitou spánku.

Správa uživatelů

- Přihlašování bude probíhat pomocí emailu se zabezpečeným ověřením.

Offline

- Aplikace bude umožňovat offline režim se synchronizací dat při obnovení připojení.

Notifikace

- Upozornění ke cvičení a spánku.

Integrace s jinými aplikacemi

- Aplikace bude umožňovat propojení s vybranou aplikací/platformou.

Prohlašuji, že jsem maturitní projekt vypracoval(a) samostatně, výhradně s použitím uvedené literatury.

V Pardubicích 16. 3. 2026 .....

(vlastnoruční podpis)

Rád bych upřímně poděkoval své vedoucí práce, Ing. Monice Borkovcové, za odborné vedení, rady, trpělivost a čas, který mi věnovala při konzultacích a zpracování tohoto maturitního projektu.

Resumé

Tato práce představuje aplikaci „LifeMeter“, určenou pro sledování a analýzu životních metrik uživatele za účelem osobního rozvoje a zlepšení zdraví. Aplikace umožňuje uživatelům centralizované zaznamenávání a monitorování stravy, kvality spánku a fyzické aktivity.

Backendová část systému je postavena na technologiích Bun, Hono a relační databázi PostgreSQL. Pro komunikaci mezi mobilní aplikací a serverem je implementováno REST API, které zajišťuje autentizaci, synchronizaci dat a správu uživatelských profilů.

Mobilní klientská aplikace je vyvinuta pomocí technologií React Native a Expo. Zvolený přístup zaručuje kompatibilitu a zároveň umožňuje přístup k nativním rozhraním pro synchronizaci zdravotních dat, konkrétně HealthKit pro operační systém iOS a Health Connect pro platformu Android.

Klíčová slova: *mobilní aplikace, zdravotní metriky, React Native, Expo, REST API, Bun, Hono, PostgreSQL*

Summary

This thesis presents the „LifeMeter“ application, designed for tracking and analyzing a user's lifestyle metrics to support personal development and health improvement. The application allows users to centrally log and monitor their diet, sleep quality, and physical activity.

The backend of the system is built on Bun, Hono, and a PostgreSQL relational database. A REST API is implemented for communication between the mobile application and the server, providing authentication, data synchronization, and user profile management.

The mobile client application is developed using React Native and Expo. The chosen approach ensures cross-platform compatibility while enabling access to native APIs for health data synchronization, specifically HealthKit for the iOS operating system and Health Connect for Android.

Keywords: *mobile application, health metrics, React Native, Expo, REST API, Bun, Hono, PostgreSQL*

Obsah

Úvod **8**

1 Teoretická část **9**

1.1 Současný stav . . . 9

1.2 Existující řešení . . . 10

1.2.1 Google Fit . . . 10

1.2.2 Apple Health . . . 10

1.2.3 MyFitnessPal . . . 10

1.2.4 Cronometer . . . 10

1.2.5 Sleep Cycle . . . 11

1.3 Volba nástrojů a zdůvodnění . . . 11

1.3.1 React Native a Expo . . . 11

1.3.2 Bun a Hono . . . 11

1.3.3 PostgreSQL . . . 12

1.3.4 Better Auth . . . 12

1.3.5 Docker a Oracle Cloud . . . 12

2 Metodika a vlastní řešení **13**

2.1 Architektura systému . . . 13

2.2 Datový model . . . 13

2.3 Popis řešení . . . 13

2.4 Návrh aplikace . . . 13

2.4.1 Use Cases . . . 13

2.4.2 UML diagram . . . 13

2.4.3 Class diagram . . . 13

2.5 Popis důležitých algoritmů . . . 13

Výsledky **14**

Diskuze **15**

Závěr **16**

Literatura **17**

Seznam obrázků **18**

Seznam tabulek **19**

Seznam příloh **21**

A Diagramy 22
B Tabulky 23
C Schemata 24
D Api dokumentace 25
E Instalační návod 26

Úvod

Sledování zdravotních metrik se v posledních letech stalo běžnou součástí každodenního života stále většího počtu lidí. Rozšíření chytrých telefonů a nositelných zařízení umožnilo sbírat data o fyzické aktivitě, výživě či spánku způsobem, který byl dříve dostupný pouze ve specializovaných zařízeních. Zájem o oblast takzvaného *quantified self* – tedy systematického zaznamenávání vlastních biologických a behaviorálních dat za účelem osobního rozvoje – neustále roste [1].

Stávající řešení na trhu (např. MyFitnessPal, Google Fit či Hevy) se často zaměřují jen na dílčí aspekty zdraví a uživatel je nucen přecházet mezi více nepropojenými aplikacemi, mnohdy s omezenými funkcemi v neplacených verzích. Cílem tohoto maturitního projektu je proto návrh a implementace aplikace *LifeMeter*, která usiluje o centralizaci sledovaných metrik (fyzické aktivity, výživa, spánek, hmotnost) do jediného prostředí s přehlednou domovskou obrazovkou.

Mobilní klient pro systémy Android a iOS je vyvinut pomocí frameworků React Native [2] a Expo [3]. Synchronizaci dat ze systémových úložišť lokálních zařízení zajišťují rozhraní Health Connect a HealthKit. Serverová část, komunikující s klientem přes REST API, je postavena na moderním běhovém prostředí Bun [4] s využitím frameworku Hono a relační databáze PostgreSQL. Celý backendový systém je kontejnerizován v prostředí Docker a nasazen na platformě Oracle Cloud.

1. Teoretická část

1.1 Současný stav

Moderní mobilní operační systémy poskytují vývojářům třetích stran standardizovaná rozhraní pro přístup ke zdravotním datům uloženým v centralizovaném systémovém úložišti. Na platformě Android tuto roli od verze Android 8 zastává Health Connect [5], na platformě iOS pak HealthKit [6], dostupný od verze iOS 8. Obě rozhraní fungují na principu zprostředkování – aplikace nezíská přímý přístup k datům jiných aplikací, nýbrž čte z a zapisuje do sdíleného úložiště, přičemž uživatel musí explicitně udělit oprávnění ke každému datovému typu zvlášť. Tento model výrazně zjednodušuje integraci mezi aplikacemi (sledování kroků, chytré hodinky, fitness tracker) a eliminuje nutnost vytvářet proprietární synchronizační protokoly.

Klíčovým modelem při návrhu mobilních aplikací závislých na vzdáleném serveru je přístup *offline-first* [7]. Mobilní zařízení jsou ze své podstaty vystavena nestabilnímu nebo zcela chybějícímu síťovému připojení, proto robustní aplikace nepředpokládá nepřetržitou konektivitu, nýbrž primárně pracuje s lokálně uloženými daty a se serverem se synchronizuje, jakmile je připojení k dispozici. Na platformě React Native je standardním řešením pro rychlé lokální úložiště knihovna MMKV [8], která využívá nativní implementaci ve formě sdílené paměti mapované na soubor a dosahuje výrazně nižší latence než `AsyncStorage`.

Komunikace mezi mobilním klientem a serverovou částí je v současných aplikacích zpravidla realizována prostřednictvím architektury REST (*Representational State Transfer*) [9]. REST definuje sadu architektonických omezení – bezstavovost, jednotné rozhraní, vrstvený systém – která vedou k interoperabilním, škálovatelným a snadno testovatelným API. Každý zdroj je identifikován URL adresou, operace nad ním jsou vyjádřeny standardními HTTP metodami (GET, POST, PUT, DELETE) a data jsou přenášena zpravidla ve formátu JSON.

Autentizace uživatelů v moderních webových a mobilních aplikacích se nejčastěji řeší dvěma přístupy. Prvním jsou tokeny JWT (*JSON Web Token*), které jsou bezstavové a snadno horizontálně škálovatelné, avšak jejich uložení na straně klienta přináší bezpečnostní rizika, pokud jsou uchovávány v přístupném úložišti. Druhým přístupem jsou

stavové relace spravované serverem s identifikátorem uloženým v `HttpOnly` cookie, která není přístupná JavaScriptovým kódem, čímž je eliminováno riziko krádeže tokenu skriptem třetí strany [10].

1.2 Existující řešení

Na trhu existuje řada aplikací zaměřených na sledování zdravotních metrik. Níže jsou popsána nejrozšířenější řešení včetně jejich omezení, jež vedla k rozhodnutí vytvořit vlastní aplikaci.

1.2.1 Google Fit

Google Fit [11] je platforma vyvinutá společností Google, integrovaná přímo do operačního systému Android. Automaticky zaznamenává pohybovou aktivitu prostřednictvím senzorů zařízení a slouží jako centrální úložiště zdravotních dat pro ekosystém Android. Aplikace nabízí jednoduché přehledy a cíle, avšak detailní sledování výživy ani spánku nativně nepodporuje – tyto oblasti vyžadují napojení na specializované aplikace třetích stran.

1.2.2 Apple Health

Apple Health [12] zastává v ekosystému iOS analogickou roli jako Google Fit na Androidu – jde o systémové úložiště zdravotních dat přístupné přes rozhraní HealthKit. Aplikace agreguje záznamy z různých zdrojů a zobrazuje je v přehledném rozhraní. Platforma je dostupná výhradně na zařízeních Apple a funguje primárně jako datový hub; vlastní zadávání výživy nebo tréninkových plánů není nativně podporováno.

1.2.3 MyFitnessPal

MyFitnessPal [13] je jednou z nejrozšířenějších aplikací pro sledování výživy na světě. Disponuje rozsáhlou databází potravin, podporou skenování čárových kódů a integrací s fitness zařízeními. Sledování tréninkového zatížení je omezeno na kalorický výdej bez možnosti podrobné analýzy silových tréninků. Velká část funkcionalit – pokročilé makroanalýzy, plánování, hloubkové statistiky – je dostupná pouze v placené verzi Premium.

1.2.4 Cronometer

Cronometer [14] se od konkurence odlišuje důrazem na nutriční přesnost. Databáze potravin vychází primárně z ověřených vědeckých zdrojů (USDA, NCCDB) a aplikace sleduje vedle makroživin také mikroživiny, vitamíny a minerály. Sledování fyzické aktivity

a spánku je v základní verzi omezeno; pokročilé funkce jsou vázány na placenou verzi Gold.

1.2.5 Sleep Cycle

Sleep Cycle [15] je aplikace specializovaná výhradně na analýzu kvality spánku. Ke sledování využívá mikrofon zařízení nebo propojení s chytrými hodinkami a na základě detekce pohybu a zvukových vzorů odhaduje spánkové fáze. Nabízí chytrý budík probouzející uživatele v nejlehčí fázi spánku. Aplikace nenabízí žádnou integraci se sledováním výživy či pohybu a pokročilé statistiky jsou dostupné pouze v rámci předplatného.

1.3 Volba nástrojů a zdůvodnění

Výběr technologií byl proveden s ohledem na požadavky projektu, a osobní znalostí s některými z technologií.

1.3.1 React Native a Expo

Pro vývoj mobilního částí byl zvolen framework React Native [2] v kombinaci s platformou Expo [3]. React Native umožňuje sestavit jedinou sdílenou kódovou základnu v jazyce JavaScript/TypeScript, ze které jsou generovány nativní aplikace pro Android i iOS. Tato vlastnost byla klíčovým kritériem výběru – projekt byl sice primárně navrhován pro Android, avšak podpora iOS je zachována pro případnou budoucí expanzi bez nutnosti přepisovat aplikaci od základu. Rozhodujícím faktorem byla také předchozí znalost ekosystému React, jež výrazně zkrátila dobu potřebnou k osvojení frameworku.

Jako alternativy připadaly v úvahu framework Flutter a nativní vývoj v jazycích Kotlin resp. Swift. Flutter sice nabízí srovnatelnou multiplatformnost, avšak vyžaduje znalost jazyka Dart, který nebylo možné sdílet se serverovou částí. Nativní vývoj by poskytl maximální přístup k systémovým API za cenu nutnosti udržovat dvě zcela oddělené kódové základny.

Expo bylo zvoleno jako současný standard React Native vývoje. Managed workflow zjednodušuje konfiguraci nativních závislostí, nástroj EAS Build zajišťuje automatizované sestavení v rámci CI/CD pipeline na GitHub Actions a Expo SDK poskytuje stabilní přístup k nativním modulům – včetně integrace s rozhraními Health Connect [5] a HealthKit [6].

1.3.2 Bun a Hono

Serverová část je postavena na rozhraní Bun [4] s využitím webového frameworku Hono [16]. Bun je moderní JavaScriptový runtime s výrazně vyšším výkonem oproti Node.js – v po-

rovnání s Node.js dosahuje násobně kratších dob startu i vyšší propustnosti HTTP požadavků [17]. Klíčovou výhodou je vestavěná podpora jazyka TypeScript bez nutnosti transpilace, což zjednodušuje vývojový proces při zachování plné kompatibility s ekosystémem npm.

Hono je odlehčený webový framework optimalizovaný pro serverová prostředí. Vyznačuje se minimálními závislostmi, typově bezpečným směrováním požadavků a přímočarým API. Kombinace Bun + Hono byla zvolena jako moderní alternativa ke standardnímu stacku Node.js + Express.

1.3.3 PostgreSQL

Jako databázový systém byla zvolena databáze PostgreSQL [18]. Datový model aplikace LifeMeter zahrnuje vzájemně provázané entity (uživatelé, jídla, tréninky, spánkové záznamy), jejichž vztahy jsou přirozeně vyjádřeny relačním schématem s cizími klíči a referenční integritou. Relační databáze představuje přirozenější volbu oproti dokumentovým databázím (MongoDB) nebo embedded řešením (SQLite), která by nebyla vhodná pro serverovou část s souběžnými přístupy více uživatelů.

1.3.4 Better Auth

Pro správu autentizace a uživatelských relací byla zvolena knihovna Better Auth [19]. Tato volba byla upřednostněna před vlastní implementací i před cloudovými řešeními typu Firebase Authentication. Better Auth je navržen s nativní podporou jazyka TypeScript a poskytuje přímočarou integraci s frameworkem Hono prostřednictvím dedikovaného adaptéru. Jako self-hosted řešení nevyžaduje závislost na infrastruktuře třetí strany, čímž je zajištěna plná kontrola nad uživatelskými daty. Autentizace je realizována prostřednictvím `HttpOnly` cookies se stavovými relacemi na serveru, v souladu s doporučeními OWASP [10].

1.3.5 Docker a Oracle Cloud

Celý backendový systém je kontejnerizován pomocí nástroje Docker Compose [20] a nasazen na instanci Oracle Cloud Infrastructure. Docker Compose umožňuje deklarativní definici vícekomponentního prostředí (API server, databáze, reverzní proxy) v jediném konfiguračním souboru, čímž zajišťuje reprodukovatelnost nasazení a jednoduchou správu závislostí mezi službami.

Oracle Cloud byl zvolen díky programu Always Free, který zahrnuje bezplatnou instanci s architekturou ARM s dostatečnými výpočetními prostředky pro provoz projektu. Na rozdíl od PaaS řešení poskytuje vlastní virtuální stroj plnou kontrolu nad konfigurací prostředí a správou uložených dat.

2. Metodika a vlastní řešení

2.1 Architektura systému

2.2 Datový model

2.3 Popis řešení

2.4 Návrh aplikace

2.4.1 Use Cases

2.4.2 UML diagram

2.4.3 Class diagram

2.5 Popis důležitých algoritmů

Výsledky

Diskuze

Závěr

Literatura

[1] M. Swan. „The Quantified Self: Fundamental Disruption in Big Data Science and Biological Discovery“. In: *Big Data* 1.2 (čvn. 2013). PMID: 27442063, s. 85–99. ISSN: 2167-6461. DOI: 10.1089/big.2012.0002. URL: https://journals.sagepub.com/doi/10.1089/big.2012.0002 (cit. 11. 03. 2026).

[2] Meta Platforms, Inc. *React Native Documentation*. 2026. URL: https://reactnative.dev/docs/getting-started (cit. 12. 03. 2026).

[3] 650 Industries, Inc. *Expo Documentation*. 2026. URL: https://docs.expo.dev/ (cit. 12. 03. 2026).

[4] Jarred Sumner. *Bun Documentation*. 2026. URL: https://bun.sh/docs (cit. 12. 03. 2026).

[5] Google LLC. *Health Connect – Android Developers Documentation*. 2024. URL: https://developer.android.com/health-and-fitness/guides/health-connect (cit. 15. 03. 2026).

[6] Apple Inc. *HealthKit – Apple Developer Documentation*. 2024. URL: https://developer.apple.com/documentation/healthkit (cit. 15. 03. 2026).

[7] Gregor Hoff a Martin Kleppmann. *Offline First – Designing for Unreliable Networks*. 2015. URL: https://offlinefirst.org/ (cit. 15. 03. 2026).

[8] Tencent. *MMKV – Fast, Persistent Key-Value Storage*. 2024. URL: https://github.com/Tencent/MMKV (cit. 15. 03. 2026).

[9] Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm (cit. 15. 03. 2026).

[10] OWASP Foundation. *OWASP Mobile Application Security Verification Standard*. 2024. URL: https://mas.owasp.org/MASVS/ (cit. 15. 03. 2026).

[11] Google LLC. *Google Fit: Activity Tracking*. 2024. URL: https://www.google.com/fit/ (cit. 15. 03. 2026).

[12] Apple Inc. *Apple Health*. 2024. URL: https://www.apple.com/health/ (cit. 15. 03. 2026).

[13] MyFitnessPal, Inc. *MyFitnessPal: Calorie Counter and Diet Tracker*. 2024. URL: https://www.myfitnesspal.com/ (cit. 15. 03. 2026).

[14] Cronometer Software Inc. *Cronometer: Track Nutrition & Count Calories*. 2024. URL: https://cronometer.com/ (cit. 15. 03. 2026).

[15] Sleep Cycle AB. *Sleep Cycle: Sleep Tracker*. 2024. URL: https://www.sleepcycle.com/ (cit. 15. 03. 2026).

[16] Yusuke Wada. *Hono Documentation*. 2024. URL: https://hono.dev/docs/ (cit. 15. 03. 2026).

[17] Paul Bratslavsky. *Bun vs Node.js: A Practical Comparison Every Developer Should Read*. 2025. URL: https://strapi.io/blog/bun-vs-nodejs-performance-comparison-guide (cit. 16. 03. 2026).

[18] The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*. 2024. URL: https://www.postgresql.org/docs/17/ (cit. 15. 03. 2026).

[19] Better Auth Contributors. *Better Auth Documentation*. 2024. URL: https://www.better-auth.com/docs (cit. 15. 03. 2026).

[20] Docker, Inc. *Docker Compose Documentation*. 2024. URL: https://docs.docker.com/compose/ (cit. 15. 03. 2026).

Seznam obrázků

Seznam tabulek

Seznam příloh

A Diagramy

B Tabulky

C Schemata

D Api dokumentace

E Instalační návod